

STAYKARO PRIVATE LIMITED

AI CALLER AGENT

Developer Handbook

Conventions, workflow, and standards for building V1

Version 1.1

Companion to the Architecture Specification and Team Execution Plan

Audience: Shivashree, Nishkala, Nihal, and any future engineer joining the project

Table of Contents

- 1. Welcome to This Handbook 4
- 2. Project Overview, in Plain Language 4
- 3. Repository & Folder Structure 5
- 4. Naming Conventions 6
 - 4.1 Files and folders 6
 - 4.2 Identifiers that flow through the whole system 6
- 5. Coding Standards 6
 - 5.1 General principles, before language-specific rules 6
 - 5.2 Python standards (services/api, voice-gateway, worker) 6
 - 5.3 Next.js / TypeScript standards (both dashboards) 7
- 6. API Design Standards 8
- 7. Error Handling 8
- 8. Logging 9
- 9. Git Workflow 10
 - 9.1 Branch naming 11
 - 9.2 Commit message rules 11
- 10. Pull Request Process 12
- 11. Code Review Checklist 12
- 12. Security Checklist 12
- 13. Testing Rules 13
- 14. Definition of Done 13

15. Working with AI Coding Tools (Claude / Codex)	13
16. Common Mistakes to Avoid	14
17. A Day in the Life: Daily Developer Workflow	15

1. Welcome to This Handbook

This handbook is the single source of truth for how we write code, name things, review pull requests, and work together on the Staykaro AI Caller Agent. It is written for a beginner developer — someone who may know how to code, but hasn't necessarily worked on a production, multi-tenant, real-time system before. If you already know a section cold, skim it. If you're new to a concept, read the boxed explanations before the code — they're there on purpose.

Three people are building this: **Shivashree** (Voice & AI), **Nishkala** (Backend, Data & RAG), and **Nihal** (Frontend, DevOps & Automation). This handbook applies to all three, regardless of which module you're touching.

HOW THIS FITS WITH THE OTHER DOCUMENTS

This handbook covers *how* we work. The **Architecture Specification** covers *what* we're building. The **Team Execution Plan** covers *who* builds which piece and in what order. The **API Specification** and **Database Design** documents go deep on their respective areas — this handbook only covers the conventions you need day-to-day.

2. Project Overview, in Plain Language

CONCEPT · What is an API?

WHAT IT IS

An API (Application Programming Interface) is a set of rules that lets two pieces of software talk to each other, even if they're written in different languages or run on different computers.

REAL-WORLD ANALOGY

Think of a restaurant. You (the customer) don't walk into the kitchen and cook your own food. You give your order to a waiter, who takes it to the kitchen and brings back your food. The waiter is the API — a fixed, predictable way to ask for something, without needing to know how the kitchen actually works.

WHY STAYKARO USES IT

Our Next.js dashboards never talk to PostgreSQL directly. They ask the FastAPI backend — through its API — for what they need, exactly like a waiter taking an order. This is why frontend and backend can be built by different people, in parallel, without either one needing to understand the other's internals.

TECHNICAL IMPLEMENTATION

Every API in this project is documented in full in the **API Specification** document: the URL, the HTTP method, what you send, what you get back, and what can go wrong.

Staykaro AI Caller Agent is a platform where different businesses ("tenants") each run their own AI phone agent. A caller dials a number, the system figures out which business owns that number, and the AI holds a real, spoken conversation with the caller — answering questions using that specific business's own knowledge base, in English, Hindi, or Telugu, and switching languages mid-call if the caller does.

The system has three moving parts you'll hear about constantly: the **voice pipeline** (the real-time call itself), the **platform backend** (accounts, data, billing, knowledge bases), and the **dashboards** (what Staykaro's team and clients actually see and click on). See the Architecture Specification, §5, for the full

system diagram.

3. Repository & Folder Structure

CONCEPT · Why do we agree on a folder structure before writing code?

WHAT IT IS

A folder structure is just an agreed layout for where different kinds of files live.

REAL-WORLD ANALOGY

Imagine three roommates sharing a kitchen with no agreement on where anything goes. Every meal starts with five minutes of "where do we keep the salt now?" A labeled, agreed layout — spices here, pans there — means everyone finds things instantly, including a guest who's never cooked in that kitchen before.

WHY STAYKARO USES IT

Three of us are writing code simultaneously, often on parts of the system that will later plug into each other. If Nishkala doesn't know where Shivashree's provider interfaces live, she can't safely add a new one. A fixed structure means everyone can find, and correctly place, code without asking.

TECHNICAL IMPLEMENTATION

We use a monorepo — one Git repository containing every part of the system — organized as shown below.

```
staykaro-ai-caller/
├── apps/
│   ├── admin-dashboard/      # Next.js -- Nihal
│   └── client-dashboard/     # Next.js -- Nihal
├── services/
│   ├── api/                 # FastAPI platform backend -- Nishkala
│   ├── voice-gateway/       # Python + Pipecat real-time pipeline -- Shivashree
│   ├── worker/              # Background jobs (ingestion, usage) -- Nishkala
│   └── integration-service/  # Async automation layer -- Nihal
├── packages/
│   ├── auth/                # JWT + RBAC -- Nishkala
│   ├── tenant/              # Tenant context + RLS helpers -- Nishkala
│   ├── database/            # Models, migrations, Redis session -- Nishkala
│   ├── providers/           # STT/LLM/TTS/Telephony interfaces -- Shivashree
│   ├── observability/       # Logging, metrics, audit -- shared
│   └── billing/              # Usage + Razorpay -- Nishkala
├── infrastructure/
│   ├── docker/              # Dockerfiles -- Nihal
│   ├── nginx/               # Reverse proxy config -- Nihal
│   └── monitoring/          # Metrics/alerts config -- Nihal
├── tests/
│   ├── unit/ integration/ voice/ rag/ tenant-isolation/ load/ failure/
├── docs/
├── docker-compose.yml
├── .env.example
└── README.md
```

Figure 3.1 -- Repository layout. This mirrors the module ownership in the Team Execution Plan, §A.

Rule: code that belongs to one person's owned module (per the Execution Plan) lives inside that module's folder. Shared code that multiple modules need — logging, tenant context, provider interfaces — lives in `packages/`, never copy-pasted into each service separately.

4. Naming Conventions

4.1 Files and folders

Item	Convention	Example
Python files/folders	snake_case	call_session.py
Python classes	PascalCase	class CallSessionManager:
Python functions/variables	snake_case	def get_tenant_context():
TypeScript/React components	PascalCase	AgentConfigForm.tsx
TypeScript variables/functions	camelCase	const fetchCallHistory = () => ...
Database tables	snake_case, plural	knowledge_chunks
Database columns	snake_case	created_at, tenant_id
Environment variables	UPPER_SNAKE_CASE	DEEPGRAM_API_KEY
Git branches	kebab-case with prefix	feature/voice-deepgram

4.2 Identifiers that flow through the whole system

These names are fixed by the Architecture Specification (§8) and must never be renamed locally, even for convenience: `tenant_id`, `agent_id`, `call_id`, `turn_id`, `language_code`. If you find yourself calling one of these something else in a new module, stop — you're about to break traceability across the whole system.

5. Coding Standards

5.1 General principles, before language-specific rules

- **Explicit over clever.** Code is read far more often than it's written. A slightly longer, obvious version beats a short, clever one nobody else can safely modify.
- **Small, single-purpose functions.** If you need the word "and" to describe what a function does, it probably does two things and should be two functions.
- **Fail loudly in development, fail safely in production.** An error should never be silently swallowed. See §9, Error Handling.
- **No premature abstraction.** Don't build a generic framework for a problem you have one example of. The provider interfaces (§17 of the Architecture Spec) are deliberately abstracted because we already know we're swapping providers — most code isn't in that position.

5.2 Python standards (services/api, voice-gateway, worker)

- Python 3.11+, fully type-hinted. A function without type hints will not pass code review.

- Formatting: `black`, import order: `isort`. Both run automatically in CI (\$11) — don't argue about formatting in review, let the tool decide.
- Async by default in the API and voice-gateway. If you're writing a function that does I/O (database, Redis, an external API call), it should be `async def`, not a blocking call inside async code.
- Pydantic models for every request/response shape and every config object — never pass raw dicts across a module boundary.

```
# GOOD -- typed, async, single purpose, explicit error
async def get_tenant_by_phone_number(
    phone_number: str, db: AsyncSession
) -> Tenant:
    result = await db.execute(
        select(Tenant).where(Tenant.phone_number == phone_number)
    )
    tenant = result.scalar_one_or_none()
    if tenant is None:
        raise TenantNotFoundError(phone_number)
    return tenant

# AVOID -- untyped, unclear what happens on failure
def get_tenant(number, db):
    t = db.query(Tenant).filter_by(phone_number=number).first()
    return t
```

Figure 5.1 -- Preferred vs. avoided Python style

5.3 Next.js / TypeScript standards (both dashboards)

- TypeScript strict mode on. No any without a comment explaining why it's unavoidable.
- Functional components with hooks only — no class components.
- Data fetching goes through a typed API client generated from the API Specification's schema, never raw `fetch()` calls scattered through components.
- Server state (API data) and UI state (form inputs, toggles) are kept separate — don't put API response data directly into local component state without a clear reason.

```
// GOOD -- typed props, explicit loading/error states
interface AgentConfigFormProps {
    agentId: string;
    onSave: () => void;
}

export function AgentConfigForm({ agentId, onSave }: AgentConfigFormProps) {
    const { data, isLoading, error } = useAgent(agentId);
    if (isLoading) return <Spinner />;
    if (error) return <ErrorBanner message={error.message} />;
    return <AgentForm agent={data} onSave={onSave} />;
}
```

Figure 5.2 -- Preferred component pattern

6. API Design Standards

Full endpoint-by-endpoint detail lives in the **API Specification** document. The rules every endpoint must follow, regardless of what it does:

- 1 Every request that touches tenant-scoped data resolves tenant context *before* touching the database (Architecture Spec §10) — never trust a `tenant_id` passed in the request body.
- 2 Every endpoint returns a consistent error shape (§9 below) — never a bare string or an unhandled 500.
- 3 Every endpoint that changes data validates its input with a Pydantic model — never trust client input.
- 4 Every endpoint is covered by at least one test that asserts a client cannot use it to reach another tenant's data, if it touches tenant-scoped data at all.

7. Error Handling

CONCEPT · Why do we standardize error handling?

WHAT IT IS

Error handling is the code that runs when something goes wrong — a database is down, a caller's request is malformed, an external API times out.

REAL-WORLD ANALOGY

Imagine calling three different customer support lines, and each one has a completely different way of telling you "we can't help with that right now": one plays static, one hangs up, one says a made-up reason. You can't build a reliable response to any of them. A single, predictable way of saying "this failed, here's why" is what lets the rest of the system react correctly.

WHY STAYKARO USES IT

The Architecture Specification's Failure Architecture (§18) requires every failure mode -- STT, LLM, TTS, RAG, Redis, PostgreSQL, telephony, automation -- to degrade in a specific, documented way. That's only possible if errors are raised, shaped, and caught consistently across all three people's code.

TECHNICAL IMPLEMENTATION

We use typed exception classes (e.g. `TenantNotFoundError`, `ProviderTimeoutError`) that carry enough structured detail to log and to turn into the correct HTTP status code -- never bare `Exception` and never a raw string message as the only signal of what happened.


```
class StaykaroError(Exception):
    """Base class for all application errors. Never raise this directly."""
    status_code = 500
    error_code = "internal_error"

class TenantNotFoundError(StaykaroError):
    status_code = 404
    error_code = "tenant_not_found"

class CrossTenantAccessAttempt(StaykaroError):
    """Raised the moment a request tries to touch another tenant's data.
    This is logged to the audit log AND alerts on-call -- it should never
    happen through normal use, only through a bug or an attack."""
    status_code = 403
    error_code = "cross_tenant_access_denied"
```

Figure 7.1 -- Base error classes

RULE

A caught exception is either handled meaningfully (retried, a documented fallback is used) or re-raised as a typed StaykaroError. `except Exception: pass` will be rejected in review, no exceptions.

8. Logging

CONCEPT · What is logging, and why not just use print statements?

WHAT IT IS

Logging is recording what a program is doing while it runs, so a human can look back afterward and understand what happened -- especially when something went wrong and nobody was watching in real time.

REAL-WORLD ANALOGY

A flight data recorder doesn't help the pilot fly the plane. It exists so that, after something unexpected happens, investigators can reconstruct exactly what occurred, in order, with timestamps. Logs are the same idea for software.

WHY STAYKARO USES IT

Every call in this system produces dozens of events across four or five different processes (telephony, voice gateway, RAG, LLM, TTS). When a caller says "the AI didn't understand me," logs are the only way to reconstruct, turn by turn, what actually happened -- which is why every log line carries `call_id` and `turn_id` (Architecture Spec §32).

TECHNICAL IMPLEMENTATION

We use structured logging (JSON, not free-text sentences), so logs can be filtered and aggregated by field, not grepped by hand.

```
logger.info(  
    "rag_retrieval_complete",  
    tenant_id=tenant_id,  
    call_id=call_id,  
    turn_id=turn_id,  
    chunks_returned=len(chunks),  
    latency_ms=elapsed_ms,  
)  
# NOT: print(f"Got {len(chunks)} chunks in {elapsed_ms}ms")
```

Figure 8.1 -- Structured vs. unstructured logging

NEVER LOG

Full caller transcripts at INFO level (PII), API keys or secrets under any circumstances, and full request/response bodies for authentication endpoints.

9. Git Workflow

CONCEPT · What is Git branching, and why do we need a workflow for it?

WHAT IT IS

Git lets multiple people edit the same codebase without overwriting each other's work. A branch is an independent line of changes that can be merged back in once it's ready.

REAL-WORLD ANALOGY

Think of it like three people editing copies of the same document, then a fourth person carefully merging everyone's edits together at the end -- instead of three people fighting over one shared document in real time, each accidentally deleting the others' sentences.

WHY STAYKARO USES IT

Shivashree, Nishkala, and Nihal are often touching related code simultaneously (e.g. Shivashree emits call events, Nishkala persists them). Branches let each person work without breaking the others' in-progress code, and Pull Requests are the checkpoint where changes are reviewed before they can affect anyone else.

TECHNICAL IMPLEMENTATION

We use three branch types: main, develop, and feature/*, described below.

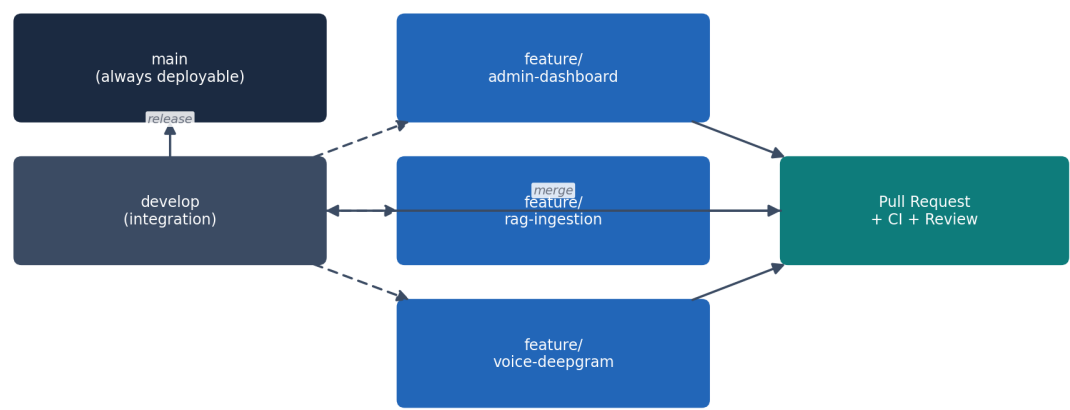


Figure 9.1 -- Branch flow. Feature branches merge into develop via reviewed PRs; develop promotes to main for release.

Branch	Purpose	Who can push directly
main	Production. Always deployable.	No one -- only via reviewed promotion from develop
develop	Integration branch. Where features land first.	No one -- only via reviewed PR
feature/*	One branch per ticket.	The ticket owner

9.1 Branch naming

Format: `feature/<short-description>`, matching the ticket it implements where possible. Examples from the Execution Plan: `feature/voice-deepgram` (SH-04), `feature/rag-ingestion` (NK-09), `feature/admin-dashboard-clients` (NH-06). For a bug fix outside a ticket: `fix/<short-description>`.

9.2 Commit message rules

Format: `<type>(<scope>): <short summary>`, imperative mood, under 72 characters on the first line.

Type	Use for
feat	A new capability
fix	A bug fix
refactor	No behavior change, code restructured
test	Adding or fixing tests only
docs	Documentation only
chore	Tooling, dependencies, config

```

feat(voice-gateway): add Deepgram streaming STT integration (SH-04)
fix(tenant): prevent stale RLS context on pooled connection reuse (NK-07)
test(rag): add multilingual retrieval matrix per architecture §16

```

Figure 9.2 -- Example commit messages, referencing their ticket IDs

10. Pull Request Process

- 1 Open the PR against `develop`, referencing the ticket ID in the title.
- 2 CI must pass -- tests, linting, security checks -- before a human review is requested (Execution Plan §M).
- 3 Reviewer is whoever is assigned per the Execution Plan's ownership rules (§L, §M) -- never the author.
- 4 For tenant-scoped or security-relevant changes, the reviewer re-runs the tenant isolation suite locally, not just trusts a green CI checkmark.
- 5 Once approved, the author merges -- reviewers approve, they don't merge on someone else's behalf.

11. Code Review Checklist

Check	Why it matters
Does this touch tenant-scoped data? If so, is <code>tenant_id</code> filtered AND is there a test proving isolation?	This is the single most consequential category of bug in the whole system.
Are all inputs validated (Pydantic / TypeScript types), not trusted?	Prevents malformed or malicious input from reaching business logic.
Are errors typed and handled per §7, not swallowed?	Silent failures are invisible until a customer complains.
Is there a test for the new behavior, and does it fail without the change?	A test that passes either way isn't testing anything.
Are secrets, API keys, or tenant data present anywhere in the diff?	Once committed, a secret is compromised even if later removed.
Does this match the naming/folder conventions in §3-4?	Consistency is what lets three people navigate one codebase quickly.
If this changes an API contract (§J of Execution Plan), did the consuming side get updated too?	A silent contract change breaks whoever depends on it, often without an obvious error.

12. Security Checklist

Before any PR touching the items below is merged, confirm against the Architecture Specification §27:

- No secret, API key, or credential is hardcoded or committed -- ever, including in test fixtures.

- Every new endpoint enforces authentication and RBAC (packages/auth) -- there is no "trusted internal" endpoint that skips this.
- Every new tenant-scoped query is covered by the tenant isolation test suite (NK-08).
- Every new webhook handler verifies its signature and is idempotent (Architecture Spec §22).
- Any new logging does not include full transcripts, secrets, or unnecessary PII (§8 above).

13. Testing Rules

Full detail on test categories lives in the **Testing Guide** document. The rule for this handbook is simpler: **a PR without a test for its new behavior does not get merged**, with one exception -- pure UI copy/styling changes with no logic. If you're not sure whether your change needs a test, it does.

14. Definition of Done

A ticket is not done because "it works locally." Matching the Team Execution Plan, §N:

- Implementation complete and matches the ticket's acceptance criteria exactly
- Unit and (where applicable) integration tests written and passing
- Errors handled per §7, logging added per §8
- Security considered -- tenant isolation re-checked if the change touches tenant-scoped data
- Documentation updated if the change affects an API contract, schema, or workflow
- Reviewed by someone other than the author, and approved
- Verified in staging, not just locally

15. Working with AI Coding Tools (Claude / Codex)

CONCEPT · Why do we need explicit rules for AI-assisted coding?

WHAT IT IS

AI coding assistants can generate working code very quickly from a description of what's needed -- but "it compiles and looks right" is not the same standard as "it's safe to run in production."

REAL-WORLD ANALOGY

Think of a very fast, very well-read junior engineer who has never actually worked at Staykaro, doesn't know our tenant isolation requirements from memory, and has never seen what happens when a webhook fires twice. Their first draft is often excellent -- but you'd never merge their code without review either.

WHY STAYKARO USES IT

The Team Execution Plan explicitly tags every ticket AI-HIGH, AI-MEDIUM, or AI-LOW (\$R) based on how much of it can be safely generated versus how much needs human judgment. Tenant-context/RLS code, licensing decisions, and endpoint tuning are AI-LOW for a reason -- they're exactly where a confident-looking wrong answer is most expensive.

TECHNICAL IMPLEMENTATION

AI-generated code goes through the exact same Definition of Done (§14) and Code Review Checklist (§11) as human-written code. There is no fast lane.

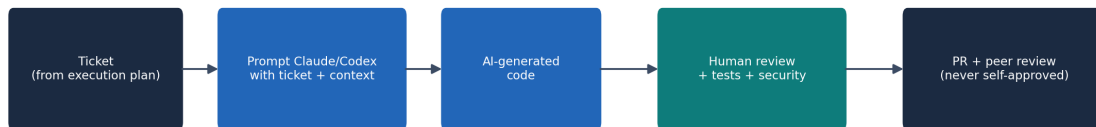


Figure 15.1 -- Where AI assistance fits: it accelerates generation, never replaces review.

- **Give the assistant real context.** Paste the relevant ticket from the Execution Plan, the relevant architecture section, and any interface it needs to conform to. A prompt with no context produces code that looks plausible but doesn't match our actual conventions.
- **Treat AI-LOW tasks as AI-LOW.** You can still use Claude/Codex to explore an approach for tenant-context propagation or endpoint tuning, but the actual implementation and the judgment behind it are yours.
- **Never paste secrets, real customer data, or real call transcripts into a prompt,** including into a coding assistant.
- **Read what was generated.** If you can't explain why a piece of generated code works, you're not ready to open a PR with it.

16. Common Mistakes to Avoid

Mistake	Why it bites you later
Filtering by tenant_id in the frontend only, not the API	The API is reachable directly -- frontend filtering is not security (Architecture Spec §26).
Catching an exception and logging it, without re-raising or handling it	The caller of your function has no idea anything went wrong.
Adding a new environment variable without updating .env.example	The next person's local setup silently breaks with no clear error.
Testing only the happy path	Production is mostly the unhappy path -- see Architecture Spec §18.
Copy-pasting a provider-specific call instead of going through the provider interface	Breaks the whole point of §17's abstraction -- now a vendor swap requires a rewrite.
Merging your own PR	Removes the one review step that catches what the author can't see in their own code.

17. A Day in the Life: Daily Developer Workflow

- 1 Pull latest devlop, check the Execution Plan for your next ticket.
- 2 Create a feature branch named after the ticket (§9.1).
- 3 Write code following the standards in §5-8, with tests as you go, not after.
- 4 Run tests and linting locally before opening a PR -- don't rely on CI to catch what you could catch in ten seconds locally.
- 5 Open the PR (§10), tag the correct reviewer per the Execution Plan's ownership tables.
- 6 Respond to review feedback the same day where possible -- a PR sitting half-reviewed blocks whatever depends on it (see the Dependency Graph, Execution Plan §G).
- 7 Once merged, check whether your change unblocks anyone else's ticket, and let them know.

IF YOU'RE BLOCKED

Check the Dependency Graph (Execution Plan §G) before assuming you're stuck -- it's common for a small, unblocking piece of someone else's ticket to be gettable early. Ask, rather than idling on a blocked ticket.